

Performance Analysis: Serial, OpenMPI, and CUDA Implementations for Matrix FFT Computation

Aditya Singh Bhadoria

June 17, 2026

1 Objective

The objective of this task was to implement and compare the computational performance of the Fast Fourier Transform (FFT) and Inverse Fast Fourier Transform (IFFT) on coherently sampled 2D sine wave matrices. The execution times were benchmarked across three paradigms: a single-threaded Serial CPU implementation, a parallelized OpenMPI CPU implementation (using 2 cores), and a CUDA-based GPU implementation.

2 Performance Results

Execution times were measured solely based on the mathematical computation phase, utilizing high-resolution chronological timers. Setup, memory allocation, and data generation times were explicitly excluded from the measurements to strictly measure mathematical throughput.

Table 1: FFT + IFFT Execution Time Across Matrix Dimensions

Matrix Size	Serial (1 CPU Core) [μ s]	OpenMPI (2 Cores) [μ s]	CUDA (GPU) [μ s]
10×10	21.431	17.879	3.282
100×100	694.088	212.666	3.841
700×700	33173.300	10029.500	3.841
2000×2000	410887.000	88204.100	3.422
5000×5000	3523320.000	516611.000	3.283

3 Observations and Analysis

3.1 Serial vs. OpenMPI Scaling

The single-threaded serial execution scales predictably as the data volume expands, exhibiting severe bottlenecks as the matrix dimensions approach 5000×5000 (approximately 25 million elements).

By introducing OpenMPI with two cores, we observe highly effective, non-linear speedups. For smaller matrices (e.g., 10×10), the parallelization yields minimal benefits because the communication overhead of the `MPI_Scatter` and `MPI_Gather` commands outweighs the actual mathematical workload. However, as the matrix scales to 5000×5000 , the OpenMPI implementation executes in a fraction of the Serial time (516,611 μ s vs. 3,523,320 μ s), proving the necessity of workload distribution for large-scale signal processing.

4 Documented Errors & Hardware Limitations

During the development and benchmarking process, several bugs, issues, and hardware constraints were identified and documented for future reference:

- **CUDA Silent Failure (Memory Limits):** A critical anomaly was observed regarding the CUDA execution times. Across all matrix scales, the GPU reported a constant execution time of approximately $3.5 \mu s$. Given hardware memory bandwidth limits, it is physically impossible to execute a forward and inverse FFT on a 5000×5000 matrix in 3 microseconds. This constant time indicates a *silent failure* within the CUDA execution pipeline. The $3.5 \mu s$ recorded strictly represents the *kernel launch overhead* (the time taken for the CPU to wake the GPU). The GPU likely encountered a memory allocation limit or required a specialized error-handling wrapper around `cufftExecZ2Z`, causing the massive mathematical execution to abort instantaneously without throwing a terminal error.
- **MPI Core Divisibility Constraints:** The OpenMPI implementation relies on evenly distributing rows across the available CPU cores. This introduced a constraint where the matrix dimension must be perfectly divisible by the number of MPI processes allocated (e.g., `dim % size == 0`). Attempting to use an arbitrary number of cores (e.g., `-np 4`) for a dimension that does not divide evenly (like 10 or 700) would result in a segmentation fault or a locked state. Code logic was added to gracefully catch this mathematical impossibility and exit early.

5 Conclusion

This comparative analysis successfully demonstrates the fundamental principles of high-performance computing. Single-core processing is sufficient only for trivial data sizes. Message Passing Interface (MPI) provides a robust, scalable solution for massive data architectures by effectively utilizing multi-core CPU environments. Finally, while GPUs offer unparalleled theoretical parallelism for operations like the Fast Fourier Transform, they introduce complex memory management challenges that must be strictly monitored to prevent silent runtime failures.